Effi

This guide explains every part of the project — the ideas, the code, the math, and the medical reasons — using simple words so anyone can understand.

TABLE OF CONTENTS

CHAPTER 1

Introduction to the Project

What Is Skin Cancer?

Your skin is the largest organ in your body. It covers everything and protects you from the sun, germs, and injury. Sometimes, skin cells grow in a wrong and uncontrolled way. This is called skin cancer.

Think of it like a factory where workers suddenly start making wrong parts and refuse to stop. Those bad parts pile up and damage the factory. That is what cancer cells do to the body.

The 8 Types This Project Studies

AK (Actinic Keratosis) — Rough skin patches from too much sun. Can become cancer.

BCC (Basal Cell Carcinoma) — Most common skin cancer. Grows slowly, rarely spreads.

BKL (Benign Keratosis-like) — Harmless rough patches that look like warts or age spots.

DF (Dermatofibroma) — Small harmless bumps, usually on legs.

MEL (Melanoma) — The most dangerous skin cancer. Can spread to other organs and kill.

NV (Nevus) — Normal moles. Usually harmless but need watching.

SCC (Squamous Cell Carcinoma) — Second most common skin cancer. Grows faster than BCC.

VASC (Vascular Lesion) — Spots made of blood vessels. Usually harmless.

Why Is Diagnosis So Hard?

Many of these conditions look very similar to the human eye — even to trained doctors. A dangerous melanoma can look almost identical to a harmless mole. Getting it wrong is life-or-death. Here is why diagnosis is difficult:

- Many types look the same — only subtle color, shape, or border differences exist.
- There are not enough skin specialists worldwide, especially in developing countries.
- Doctors can disagree with each other on the same image.
- Rare cancers are seen so rarely that even experts have limited practice identifying them.
- Lighting, camera quality, and image artifacts make images inconsistent.

Why AI Helps

AI does not get tired. It does not forget. It can look at 10,000 images per second and find tiny patterns a human eye might miss after hours of work. Most importantly, once trained properly, an AI model gives the same answer every time — no bias, no fatigue.

Quick Tip

The goal of this project: Train an AI to look at a skin lesion photo (and patient details like age and where on the body it is) and correctly identify which of 8 types it is — especially the dangerous ones.

CHAPTER 2

Fundamentals of Artificial Intelligence

What Is Artificial Intelligence?

Artificial Intelligence (AI) means making computers do things that normally require human thinking — like understanding pictures, reading text, or making decisions.

AI is the big idea. Machine Learning is one way to build it. Deep Learning is an even more powerful way.

Artificial Intelligence	The big umbrella — making machines think.
Machine Learning	Teaching machines by showing them examples.
Deep Learning	Using layered brain-like networks to learn from data.
Computer Vision	Teaching machines to understand images and video.
Medical Imaging AI	Applying computer vision to X-rays, scans, and skin photos.

How Does Machine Learning Work?

Instead of writing rules by hand (like "if the spot is dark and round, it is a mole"), you show the machine thousands of examples with correct labels. The machine finds patterns on its own.

- **Step 1 — Collect data:** Gather thousands of labeled skin images.
- **Step 2 — Feed to model:** Show the model each image and its correct label.
- **Step 3 — Make prediction:** The model guesses a label.
- **Step 4 — Check the error:** Compare the guess to the correct label.
- **Step 5 — Learn from error:** Adjust the model's settings to reduce the error.
- **Step 6 — Repeat:** Do this millions of times until errors are small.

What Is Classification?

Classification means putting things into groups. Binary classification = two groups (cat or dog). Multi-class classification = many groups (AK, BCC, BKL, DF, MEL, NV, SCC, VASC). This project does multi-class classification — 8 possible answers.

CHAPTER 3

Neural Networks — How They Work

The Idea Behind Neural Networks

A neural network is inspired by the human brain. Your brain has billions of neurons (brain cells) connected to each other. When you see something, signals travel through neurons and your brain figures out what it is. A neural network is a computer version of this idea — but much simpler.

What Is One Artificial Neuron?

One neuron takes a few numbers as input, multiplies each by a weight (importance), adds them all up, adds a bias (a constant adjustment), and then passes the result through an activation function.

```
output = activation( (x1 x w1) + (x2 x w2) + ... + (xn x wn) + bias )

                    or more compactly:

                output = activation( W . X + b )
```

x = inputs | W = weights | b = bias | W.X = dot product (multiply and add)

What Are Weights and Biases?

Think of weights like volume knobs. A high weight means that input matters a lot. A low weight means it barely matters. The bias is like a default starting point — it shifts the output up or down.

Simple Example

Imagine a neuron deciding if a lesion is dangerous:

Input 1 = dark color (x1=0.9) x weight 0.8 = 0.72

Input 2 = irregular border (x2=0.7) x weight 0.6 = 0.42

Input 3 = patient age 70 (x3=0.7) x weight 0.5 = 0.35

Sum = 0.72 + 0.42 + 0.35 + bias = 1.49 + 0.1 = 1.59

Pass through activation → output = 0.83 (83% likely dangerous)

Activation Functions

Without an activation function, a neural network is just doing simple math (addition and multiplication). It could not learn complex patterns. Activation functions add non-linearity — they let the network learn curves, not just straight lines.

ReLU	`max(0, x)`	If input is negative, output 0. Otherwise pass it through. Simple and fast. Used in most layers.
Sigmoid	`1 / (1 + e^(-x))`	Squashes any number to between 0 and 1. Used for binary (yes/no) decisions.
Softmax	`e^(xi) / sum(e^(xj))`	Turns a list of numbers into probabilities that add up to 1. Used in the final layer for multi-class prediction.
GELU	`x * P(X <= x)`	Smooth version of ReLU. Used in modern transformers. More flexible than ReLU.

Forward Propagation

Forward propagation is the journey of data through the network from start to finish.

- Data enters the first layer.
- Each neuron computes its output.
- Outputs become inputs for the next layer.
- This continues until the final layer.
- The final layer outputs a prediction (e.g., 8 probabilities for 8 classes).

Backpropagation and Learning

Backpropagation is how the network learns from its mistakes. After a wrong prediction, the error travels backwards through the network. Each weight is updated slightly to reduce that error next time.

```
Loss = how wrong the prediction was

Gradient = direction and size of the error for each weight

New Weight = Old Weight - (learning rate x gradient)

w = w - (lr x dL/dw)
```

lr = learning rate (how big each correction step is) | dL/dw = gradient of loss with respect to weight

Quick Tip

Think of it like learning to ride a bike. You fall (make an error). You figure out what you did wrong (calculate gradient). You correct your balance slightly (update weights). Repeat until you ride smoothly.

CHAPTER 4

Convolutional Neural Networks (CNNs)

Why Normal Networks Fail on Images

A normal neural network treats every pixel as a separate input. A 224x224 image has 150,528 pixels x 3 colour channels = 451,584 inputs. That is too many. And the network does not know that nearby pixels are related. CNNs solve this by using a smarter approach: sliding filters.

What Is Convolution?

Convolution is a mathematical operation where a small filter (also called a kernel) slides across an image, one small patch at a time, and multiplies then sums the values.

Simple Example: 3x3 Edge-Detection Filter

Imagine your image is a 5x5 grid of numbers. Your filter is 3x3.

The filter slides over every 3x3 patch of the image.

At each position: multiply filter values with image values, then add them all up.

The result is one number. After sliding everywhere, you get a new (smaller) grid — the feature map.

An edge-detecting filter produces HIGH values where edges exist in the image and LOW values where there are no edges.

```
Feature Map[i,j] = sum over (m,n) of: Filter[m,n] x Image[i+m, j+n]
```

This is computed for every position (i,j) in the output feature map.

Key CNN Components

Filters / Kernels	Small grids (e.g. 3x3) that detect patterns. Early filters detect edges. Later filters detect complex shapes like eyes or lesion borders.
Feature Map	The output after applying a filter to an image. Each filter creates one feature map.
Stride	How many pixels the filter moves each step. Stride 1 = move 1 pixel. Stride 2 = move 2 pixels (produces smaller output).

Padding	Adding a border of zeros around the image so the filter can work on edge pixels without shrinking the image.
Pooling	Reduces the size of feature maps. Max pooling takes the highest value in each region. Keeps the most important information.
ReLU	Applied after convolution. Removes negative values. Adds non-linearity so the network can learn complex patterns.

How CNNs Learn Hierarchically

This is the most important idea in CNNs. Each layer learns something more complex than the last:

Layer 1 (earliest): Simple edges — horizontal, vertical, diagonal lines

Layer 2: Textures — repeated patterns, dots, stripes

Layer 3: Shapes — circles, blobs, boundary curves

Layer 4: Parts — skin patterns, hair follicles, lesion colours

Layer 5+ (deep): Abstract features — the kind of lesion, its subtype

Final layers: Class probabilities — AK, MEL, NV, etc.

Why CNNs Work for Skin Lesions

Skin lesions have visual patterns that CNNs are perfect for detecting:

- Irregular borders — detected by edge-finding filters
- Colour variation — detected by colour-sensitive filters in RGB channels
- Texture differences — detected by texture filters in middle layers
- Shape irregularity — detected by shape-sensitive deep layers
- Asymmetry — captured by comparing left/right halves across feature maps

CHAPTER 5

The ISIC 2019 Dataset

What Is ISIC 2019?

ISIC stands for International Skin Imaging Collaboration. It is a group of doctors and researchers who collected thousands of dermoscopic skin photos and labelled each one. The 2019 version has about 25,000 images across 8 disease classes.

Dataset Files

Image folders — one folder per class (AK/, BCC/, BKL/, etc.), each full of JPG photos.

ISIC_2019_Training_GroundTruth.csv — a table where each row is an image name, and there are 8 columns (one per class). A 1.0 in a column means that is the correct class.

ISIC_2019_Training_Metadata.csv — a table with extra patient information: age, sex, and where on the body the lesion is.

What Is Dermoscopy?

A dermoscope is a special magnifying tool that doctors press against the skin. It removes surface reflections and lets you see into the deeper skin layers. The photos produced are much more informative than regular photos — you can see structures invisible to the naked eye.

Class Distribution — The Imbalance Problem

Not all 8 classes have the same number of images. NV (moles) has thousands of examples. VASC (vascular lesions) has only a few hundred. This is a major problem because the AI tends to just learn about the common classes and ignore the rare ones.

NV — Melanocytic Nevi	~12,000+	Majority	Harmless moles — very common
MEL — Melanoma	~4,500	Medium	Most dangerous — critical to detect
BCC — Basal Cell Carcinoma	~3,300	Medium	Common cancer — needs treatment
BKL — Benign Keratosis	~2,600	Medium	Harmless rough patches
AK — Actinic Keratosis	~900	Minority	Pre-cancerous — can progress
SCC — Squamous Cell	~600	Minority	Second most dangerous cancer

DF — Dermatofibroma	~240	Minority	Harmless but rare in dataset
VASC — Vascular	~250	Minority	Rare — model struggles with these

Metadata Columns

age_approx	Patient's age in years. Older age increases risk of BCC, SCC, and MEL.
sex	Patient sex (male/female). Some cancers are more common in one sex.
anatom_site_general	Where on the body the lesion is (back, face, leg, etc.). This matters — VASC often appears on extremities.
lesion_id	An ID to link images of the same lesion. Often missing (NaN).

Train / Validation / Test Split

We divide the data into three groups before training. We use stratified splitting — each group has the same proportion of each class so minority classes are not accidentally missing from any group.

Training set = 70% — model learns from this

Validation set = 15% — check progress during training (not used for learning)

Test set = 15% — final honest evaluation after training is fully done

CHAPTER 6

Exploratory Data Analysis (EDA)

What Is EDA?

EDA means looking at your data before doing any AI. It is like checking the ingredients before you cook. You want to know what you have, what is missing, what looks wrong, and what patterns exist.

What We Check in EDA

Class distribution bar chart: Shows how many images each class has. Reveals imbalance immediately.

Missing value count: Tells us how many metadata values are empty (NaN). We need to handle these.

Age distribution per class: Reveals medical patterns — e.g., BCC is more common in older patients.

Sex distribution per class: Shows gender bias in data. MEL is slightly more common in males.

Image samples visualisation: Lets us see what the images look like. Are they blurry? Do they have hair artifacts?

Correlation between age and class: Shows if age is predictive for certain diseases (useful for metadata fusion).

Key Insights from EDA in This Project

What EDA Revealed

Imbalance ratio of about 50:1 between NV (majority) and DF/VASC (minority).

437 images have missing age. 2,631 have missing anatomical site. Need imputation strategy.

Mean patient age is 54 years. NV peaks in younger patients. BCC and SCC peak in older patients.

Males slightly outnumber females in the dataset.

Anterior torso (chest/belly area) is the most common lesion location.

Quick Tip

EDA is not optional. If you skip it, you may train a model without knowing your data has huge problems — like 90% of images being from one class, or key columns being mostly empty.

CHAPTER 7

Image Preprocessing

Why Preprocess Images?

Raw dermoscopic images are not ready for a neural network. They have different sizes, different lighting, and often contain things that confuse the model like hair, ruler marks, and dark corners. Preprocessing fixes these problems.

Basic Preprocessing Steps

Resize to 224x224	All images must be the same size before going into the network. 224x224 pixels is the standard input size for most modern CNN models.
RGB conversion	Images come in BGR format (OpenCV default) but networks expect RGB. We convert so colours are correct.
Normalise pixels	Pixel values are 0 to 255. We divide by the mean and standard deviation of ImageNet (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) to match what pretrained models expect.

Advanced Preprocessing: Hair Removal

Many dermoscopic photos have body hair crossing the lesion. Hair looks like dark lines and confuses the model — it might learn that dark lines = danger. We remove hair using a technique called black-hat morphological filtering.

How Hair Removal Works (Step by Step)

Step 1: Convert the image to grayscale (black and white).

Step 2: Apply a 'black-hat filter' — a morphological operation that finds dark structures (like hair) on a bright background.

Step 3: Create a mask — mark all pixels where hair was detected.

Step 4: Use inpainting — fill in the hair pixels using the surrounding skin colour, like Photoshop's healing brush.

Result: Clean image without hair distracting the model.

Advanced Preprocessing: CLAHE

CLAHE stands for Contrast Limited Adaptive Histogram Equalisation. It improves the contrast of the image, especially in dark or low-contrast areas where lesion details hide.

Normal contrast enhancement makes the whole image brighter, which can wash out details or amplify noise. CLAHE works in small local patches (tiles) and limits how much it boosts contrast — so it enhances detail without creating artefacts.

Regular histogram equalisation: maps all pixel values globally

CLAHE: divides image into tiles, equalises each tile separately,

then blends neighbouring tiles — local contrast without over-amplification

CHAPTER 8

Data Augmentation

What Is Overfitting?

Overfitting is when the model memorises the training data instead of learning general patterns. It gets 99% accuracy on training images but fails on new ones. It is like a student who memorises all the exam answers but cannot answer any slightly different question.

How Augmentation Fixes This

Augmentation creates new training examples by applying random changes to existing images. Each time an image is shown to the network, it looks slightly different. This forces the network to learn the actual features of the lesion, not the exact pixels.

Augmentations Used in This Project

RandomResizedCrop	Randomly crops and zooms into the image. Forces model to recognise lesions at different scales.
HorizontalFlip	Flips the image left-right (50% chance). Lesions look the same both ways — doubles effective data.
VerticalFlip	Flips the image up-down (50% chance). Same idea as horizontal flip.
ShiftScaleRotate	Moves, zooms, and rotates the image by random amounts. Rotation up to 360 degrees.
ColorJitter	Randomly changes brightness, contrast, saturation, and hue. Makes model robust to different cameras.
CLAHE	Applied randomly (30% chance) during training for extra contrast variation.
GridDistortion	Warps the image in a grid-like pattern. Simulates how skin stretches.
ElasticTransform	Applies wave-like warping. Simulates natural skin deformation.
CoarseDropout	Randomly removes rectangular patches from the image. Prevents model from relying on any one area.

MixUp Augmentation — A Special Technique

MixUp is a modern technique not in the original thesis. It creates completely new training examples by blending two images together.

```
Mixed Image = lambda x Image_A + (1-lambda) x Image_B

lambda is drawn from a Beta distribution (e.g., Beta(0.4, 0.4))

The label is also mixed: lambda% chance of label_A, (1-lambda)% chance
                         of label_B
```

The model learns to be uncertain and smooth in its predictions, which prevents overconfidence and improves minority class performance.

Test-Time Augmentation (TTA)

During testing, instead of making one prediction per image, we make 4 predictions (original, flipped, brightness-adjusted, vertically flipped) and average them. This is like getting a second and third opinion — the average prediction is more reliable than any single one.

CHAPTER 9

The Class Imbalance Problem

Why Imbalance Is Dangerous in Medicine

Imagine you have 100 patients. 95 are healthy. 5 have cancer. A lazy model can get 95% accuracy by saying everyone is healthy. But it would miss all 5 cancer patients. In medicine, that is catastrophic.

> **The Real Danger**
>
> Missing a melanoma (false negative) = patient does not get treatment = cancer spreads = possible death.
>
> Flagging a mole as melanoma (false positive) = unnecessary biopsy = stress and cost.
>
> False negatives are far more dangerous in medical AI. We care most about Recall (sensitivity).

Three Strategies to Fight Imbalance

Strategy 1: WeightedRandomSampler

When building each training batch, we pick samples with higher probability for rare classes. If VASC has 250 examples and NV has 12,000 examples, each VASC image is 48x more likely to be picked for a batch. This ensures the model sees rare classes often during training.

```
sample_weight[i] = 1 / count_of_class[i]

VASC weight = 1/250 = 0.004 (high — picked often)

NV weight = 1/12000 = 0.000083 (low — picked less often)
```

Strategy 2: Class Weights in Loss Function

We give higher penalty to mistakes on rare classes. If the model misclassifies a VASC image, the loss is much higher than if it misclassifies an NV image. This pushes the model to focus harder on rare classes.

Strategy 3: Label Smoothing

Label smoothing is a new technique added beyond the thesis baseline. Instead of training the model to output exactly 1.0 for the correct class, we soften the target to 0.9 for correct and 0.01 spread across other classes. This prevents the model from being overconfident about majority classes.

Normal label: [0, 0, 0, 0, 1, 0, 0, 0] (100% confident)

Smooth label: [0.013, 0.013, ..., 0.9, ..., 0.013] (90% confident)

Formula: smooth_label = (1-epsilon) x one_hot + epsilon / num_classes

epsilon = 0.1 (smoothing factor)

CHAPTER 10

Focal Loss — Full Mathematical Explanation

Step 1: Understanding Cross-Entropy Loss

Cross-entropy is the standard loss function for classification. It measures how different the model's predicted probabilities are from the true label.

```
CE Loss = -log( p(correct class) )

If model predicts 0.9 for the correct class: loss = -log(0.9) = 0.105
(small, good)

If model predicts 0.1 for the correct class: loss = -log(0.1) = 2.303
(large, bad)
```

Step 2: Why Cross-Entropy Fails on Imbalanced Data

The model sees 12,000 NV images for every 250 VASC images. After a while, the loss from the many NV images dominates. The model becomes excellent at NV and ignores VASC. Easy examples (NV that are obviously moles) keep contributing large amounts to the total loss, drowning out the rare hard examples.

Step 3: How Focal Loss Fixes This

Focal Loss adds a modulating factor (1 - p_t) raised to a power gamma. When the model is already confident about an easy example (p_t is high), this factor becomes tiny — the loss contribution shrinks. When the model is uncertain about a hard example (p_t is low), the factor is close to 1 — the full loss is preserved.

Focal Loss = -alpha x (1 - p_t)^gamma x log(p_t)

p_t = model's predicted probability for the correct class

gamma = focusing parameter (typically 2.0)

alpha = class weight (higher for rare classes)

Example with gamma=2:

Easy example (p_t=0.9): loss = -(1-0.9)^2 x log(0.9) = 0.01 x 0.105 = 0.001

Hard example (p_t=0.1): loss = -(1-0.1)^2 x log(0.1) = 0.81 x 2.30 = 1.864

The hard example (p_t=0.1) gets 1864x more training signal than the easy example (p_t=0.9). The model focuses its learning where it actually needs improvement.

Medical Impact of Focal Loss

With standard Cross-Entropy, the model may achieve high overall accuracy but miss most MEL and SCC cases.

With Focal Loss + class weights, the model is forced to learn the rare dangerous cancers properly.

Recall for MEL, SCC, and VASC improves significantly. This is life-saving in practice.

CHAPTER 11

Metadata Fusion and Multimodal Learning

What Is Multimodal Learning?

Multimodal means using multiple types of data together. In this project, each patient has both an image AND clinical information. A real doctor does not just look at the photo — they also consider the patient's age, sex, and where the lesion is. Multimodal learning teaches the AI to do the same.

Why Metadata Matters Clinically

Age: BCC and SCC are far more common in people over 60. MEL peaks in 50-70 age group.

Sex: MEL is slightly more common in males. Some studies show hormonal influences on BKL.

Anatomical site: VASC lesions mostly appear on hands and feet. MEL most commonly on the back (men) or legs (women).

Combining this with image features gives the AI a clinical picture, not just a visual one.

How Metadata Is Encoded

Neural networks only understand numbers. We convert each metadata field into numbers using specific techniques:

age_approx	Normalised using StandardScaler: subtract mean age (54), divide by std (18). Result: a number near 0 for average age.
sex	One-Hot Encoded into 3 columns: [female=1,male=0,unknown=0] or [female=0,male=1,unknown=0]. Missing → 'unknown'.
anatom_site_general	One-Hot Encoded into 9 columns (8 sites + unknown). Each site becomes its own 0/1 feature.

Total metadata feature vector size: 1 + 3 + 9 = 13 numbers per patient.

The Dual-Branch Architecture

The model has two separate paths that then combine. Think of it as two experts who each study different information, then compare notes and make a joint decision.

Architecture Flow

IMAGE BRANCH: Image (224x224x3) → EfficientNetV2-S → AdaptiveAvgPool → 1280-dimensional feature vector

METADATA BRANCH: Metadata (13 numbers) → Linear(13→64) → BatchNorm → ReLU → Dropout → Linear(64→128) → 128-dimensional vector

FUSION: Concatenate [1280 + 128 = 1408 numbers] → Linear(1408→512) → BN → ReLU → Dropout → Linear(512→256) → Linear(256→8)

OUTPUT: 8 probabilities (one per class). Softmax gives the final prediction.

Why Concatenation Fusion Works

Concatenation is the simplest and most effective fusion strategy. By joining the 1280-d image features with the 128-d metadata features into one 1408-d vector, the subsequent dense layers can learn complex interactions between the two modalities. For example, the network might learn: if image looks like MEL AND patient is over 60 AND lesion is on the back → very high MEL probability.

CHAPTER 12

Modern CNN Architectures

Why Not Just Use a Simple CNN?

A simple CNN from scratch would take months of data and training to reach acceptable accuracy. Modern architectures were designed by teams of researchers, trained on millions of images, and have already learned incredibly powerful feature detectors. We use transfer learning — take these powerful networks and fine-tune them for skin cancer.

EfficientNetV2-S — The Primary Backbone

EfficientNetV2-S is the model chosen as the main backbone for this project. It was designed by Google and is one of the most efficient and accurate models available.

Why EfficientNetV2-S Was Chosen

21 million parameters — powerful but not too large for medical GPU budgets.

Pretrained on ImageNet (1.28M images, 1000 classes) — already understands textures, edges, and shapes.

Uses MBConv (Mobile Inverted Bottleneck Convolution) blocks — very efficient feature extraction.

Uses Fused-MBConv in early layers — faster computation while maintaining accuracy.

Progressive training approach (trains at small image sizes first, then larger) — more stable.

Achieves state-of-the-art accuracy on standard benchmarks while being faster than older models.

MBConv Blocks Explained Simply

An MBConv block (Mobile Inverted Bottleneck) has three steps:

Expand: A 1x1 convolution multiplies the number of channels (e.g., 32 → 192) to create a rich feature space.

Depthwise Convolution: A 3x3 convolution is applied separately to each channel. Much cheaper than normal convolution.

Project: A 1x1 convolution reduces channels back (e.g., 192 → 32). Keeps the model slim.

Squeeze-and-Excitation: Recalibrates channel weights — tells the block which channels matter most.

Skip Connection: Adds the input directly to the output (if same shape). Helps gradients flow during training.

ConvNeXt-Tiny — The Second Backbone

ConvNeXt was designed by Facebook AI in 2022. It is a pure CNN but uses ideas from transformers — like LayerNorm, depthwise convolutions with large 7x7 kernels, and GELU activation. It achieves transformer-level accuracy with the simplicity of a CNN.

Transfer Learning — Using Pretrained Weights

The key idea: a network trained on 1.28 million photos of everyday objects has already learned to detect edges, textures, shapes, and colours. These low-level features are useful for ANY image task, including skin lesions. We load these pretrained weights and then fine-tune on our skin cancer dataset.

```
ImageNet training: learns general visual features

Our fine-tuning: specialises those features for skin lesions

Result: much better accuracy than training from random weights
```

CHAPTER 13

Attention Mechanisms

What Is Attention?

When you look at a skin lesion photo, you do not look at every pixel equally. Your eyes focus on the important parts — the lesion border, the colour variation, the irregular shape. Attention mechanisms teach neural networks to do the same.

Squeeze-and-Excitation (Channel Attention)

Channel attention asks: which feature detectors (channels) are most important for this specific image? It learned to amplify important channels and suppress unimportant ones.

How SE Attention Works (Step by Step)

Step 1 — Squeeze: Apply Global Average Pooling to collapse each channel to a single number. (e.g., 512 channels → 512 single numbers)

Step 2 — Excitation: Pass through a small neural network [512 → 64 → 512] with sigmoid output. Each channel gets a score between 0 and 1.

Step 3 — Scale: Multiply each channel's feature map by its score. High-score channels become louder. Low-score channels become quieter.

Result: The network focuses on the most diagnostic features for that specific image.

```
Squeeze: s = GlobalAvgPool(X) [shape: C x 1 x 1]

Excitation: e = sigmoid( W2 x ReLU( W1 x s ) )

Scale: X_out = X x e (element-wise channel multiplication)
```

The Original SA-Net (Thesis Baseline)

The original thesis used a custom SA-Net with four convolutional stages (64→128→256→512 channels) and a Skin Attention Block at each stage. The Skin Attention Block was a variation of Squeeze-and-Excitation with a reduction factor of 8. In this updated project, SA-Net is kept only as the baseline for comparison (Experiment 1). All new experiments use modern pretrained backbones which incorporate better attention mechanisms.

CHAPTER 14

Vision Transformers (ViT)

What Is a Transformer?

Transformers were originally invented for language translation (Google's BERT, GPT). The key idea: instead of processing information step by step (like a CNN layer by layer), look at ALL parts at the same time and decide which parts are relevant to each other.

How Vision Transformers Work

Patch Splitting	The 224x224 image is cut into small 16x16 patches (196 patches total). Each patch is like a 'word' in a sentence.
Linear Embedding	Each 16x16 patch (768 numbers) is projected to a smaller embedding vector.
Positional Encoding	Add position information so the model knows where each patch came from in the image.
Self-Attention	Every patch 'looks at' every other patch and decides how relevant they are to each other. A dark patch near an irregular border might highly attend to all other dark patches.
Multi-Head Attention	Run self-attention multiple times in parallel (8 or 16 'heads'), each looking at different aspects. Then combine the results.
Feed-Forward Layers	After attention, process each patch with a small neural network independently.
Classification Head	Take the special [CLS] token output and pass through a linear layer to get class probabilities.

CNN vs Transformer — Key Difference

CNN vs ViT Comparison

CNN: Builds understanding locally first (small patches) then globally (large areas). Good for texture and local patterns. Fast with limited data.

ViT: Looks at global relationships from the very beginning. Every patch communicates with every other patch. Better at long-range dependencies. Needs more data to train from scratch.

This project uses EfficientNetV2-S (CNN-based) as the primary backbone because it performs better on the ISIC 2019 dataset size with pretrained transfer learning.

CHAPTER 15

The Training Pipeline

Overview of One Training Epoch

One epoch means the model has seen every training image once. Training runs for multiple epochs (25-30 in this project). Here is what happens inside each epoch:

1. **DataLoader builds batches:** Groups images into batches of 64. Applies augmentation. Transfers to GPU memory.
2. **Forward pass:** Each batch goes through the network. Output: 64 rows of 8 probabilities.
3. **Compute loss:** Compare predictions to true labels using Focal Loss. Get one loss number.
4. **Backward pass (backpropagation):** Compute gradient of loss with respect to every weight in the network.
5. **Gradient clipping:** If any gradient is too large (>1.0), scale it down. Prevents unstable training.
6. **Optimizer step:** AdamW updates all weights using gradients.
7. **Learning rate schedule:** CosineAnnealingWarmRestarts adjusts the learning rate.
8. **Repeat for all batches:** With batch size 64 and ~17,000 training images, there are ~265 batches per epoch.

The AdamW Optimiser

AdamW (Adam with Weight Decay) is the standard optimiser for modern deep learning. It adapts the learning rate for each parameter individually based on past gradients.

```
m_t = beta1 x m_(t-1) + (1-beta1) x gradient [momentum — smoothed gradient]

v_t = beta2 x v_(t-1) + (1-beta2) x gradient^2 [velocity — tracks gradient size]

w = w - lr x (m_t / sqrt(v_t) + epsilon) - lr x lambda x w

lambda = weight decay (0.0001 in this project)
```

CosineAnnealingWarmRestarts — Learning Rate Schedule

The learning rate is not fixed. It follows a cosine curve — starting at the set value, gradually decreasing to near zero, then jumping back up (warm restart). This helps escape local minima and find better solutions.

```
lr(t) = eta_min + 0.5 x (eta_max - eta_min) x (1 + cos(pi x t / T_0))

T_0 = 8 epochs (first restart cycle length)

T_mult = 2 (each subsequent cycle is 2x longer)
```

Early Stopping

If the validation Macro F1 score does not improve for 6 consecutive epochs, we stop training and restore the best weights. This prevents overfitting and saves compute time.

Mixed Precision Training (AMP)

Normally, all calculations use 32-bit floating point numbers (FP32). AMP (Automatic Mixed Precision) uses 16-bit numbers (FP16) where possible. This halves memory usage and is 2-3x faster on modern GPUs (like T4 and A100). A GradScaler prevents underflow — it scales losses up before the backward pass and scales gradients back down before the optimiser step.

CHAPTER 16

GPU Training and CUDA

Why GPU Matters So Much

Training a deep learning model involves hundreds of millions of multiplications and additions. A CPU (your regular processor) does these one at a time, or a few in parallel. A GPU (graphics processor) has thousands of small cores designed to do these all at once in parallel.

CPU speed	Typical: 10-100 batches per second for DL. Excellent for general tasks.
GPU speed	Typical: 200-2000 batches per second for DL. Designed for parallel math.
Training time	A model that takes 24 hours on CPU might take 30 minutes on GPU.
CUDA	NVIDIA's programming language for GPU. PyTorch uses CUDA automatically.
VRAM	GPU's own memory (video RAM). T4 has 16GB. Limits batch size.

GPU Optimisations in This Project

torch.compile: PyTorch 2.0 feature. Analyses the model's computation graph and fuses operations into optimised GPU kernels. 20-40% speedup.

cuDNN benchmark mode: Automatically selects the fastest convolution algorithm for your specific GPU and input sizes.

TF32 precision: On Ampere GPUs (A100, RTX30xx), TF32 gives near-FP32 accuracy at FP16 speed.

non_blocking GPU transfers: CPU-to-GPU data transfer overlaps with GPU computation. No waiting.

pin_memory in DataLoader: Keeps data in page-locked memory so GPU can grab it instantly without copying.

persistent_workers: DataLoader workers stay alive between epochs instead of restarting — saves startup time.

fused AdamW: A GPU-native optimiser that updates all parameters in one fused kernel call. Faster than standard.

Batch size 64: Larger batches use GPU more efficiently (less idle time). Doubled from the CPU-safe 32.

CHAPTER 17

Evaluation Metrics

The Confusion Matrix — The Foundation

Everything comes from the confusion matrix. It shows how many predictions were correct and how many were wrong, and exactly which classes got confused with which.

Confusion Matrix Terms (for one class vs rest)

True Positive (TP): Model said MEL. Actually was MEL. CORRECT.

True Negative (TN): Model said not-MEL. Actually was not-MEL. CORRECT.

False Positive (FP): Model said MEL. Actually was not-MEL. WRONG (over-alarm).

False Negative (FN): Model said not-MEL. Actually was MEL. WRONG (missed cancer).

In medicine: FN is worse than FP. Missing cancer is more dangerous than a false alarm.

Accuracy	`(TP+TN) / (TP+TN+FP+FN)`	% of all predictions correct. Misleading on imbalanced data.
Precision	`TP / (TP+FP)`	Of all MEL predictions, how many were actually MEL. Quality of positive predictions.
Recall / Sensitivity	`TP / (TP+FN)`	Of all actual MEL cases, how many did we find. MOST IMPORTANT for cancer detection.
Specificity	`TN / (TN+FP)`	Of all non-MEL cases, how many did we correctly identify as non-MEL.
F1-Score	`2 x (Precision x Recall) / (Precision + Recall)`	Harmonic mean of precision and recall. Balance between the two.
Macro F1	`Average F1 across all classes (equal weight)`	Treats each class equally. Penalises poor performance on rare classes.
Weighted F1	`Average F1 weighted by class size`	Gives more weight to common classes. Better reflects overall real-world performance.
ROC-AUC	`Area under the ROC curve`	Measures how well model separates classes at all threshold levels. 1.0 = perfect, 0.5 = random.

Quick Tip

For skin cancer AI, Recall (Sensitivity) for MEL and SCC is the most critical metric. A model with 80% accuracy but 30% MEL recall is dangerous — it misses 70% of melanomas. Always report per-class recall for minority classes.

CHAPTER 18

Grad-CAM Explainability

The Black-Box Problem in Healthcare

A doctor would not trust a diagnosis from a system that just says 'I think it is melanoma' without explaining why. AI models are often black boxes — they give an answer but no reason. This is a major barrier to clinical adoption. Explainable AI (XAI) solves this.

What Is Grad-CAM?

Grad-CAM stands for Gradient-weighted Class Activation Mapping. It creates a heatmap that highlights which areas of the image most influenced the model's decision. Red areas = very important for the decision. Blue areas = less important.

How Grad-CAM Works — Step by Step

Step 1: Choose a target class: We want to know why the model predicted MEL (for example).

Step 2: Forward pass: Run the image through the network. Get the prediction.

Step 3: Compute gradients: Calculate how much each activation in the last conv layer influenced the MEL score.

Step 4: Global average pooling: For each feature map (channel), compute the mean gradient across all spatial positions. This is the weight alpha_k for that channel.

Step 5: Weighted sum: Multiply each feature map by its weight and add them all together.

Step 6: ReLU: Apply ReLU to keep only positive influences (what activated the prediction, not what suppressed it).

Step 7: Upsample: Scale the result back up to the original image size. Overlay as a coloured heatmap.

```
alpha_k = (1/Z) x sum over (i,j) of: dY^c / dA^k_(i,j)

L_Grad-CAM = ReLU( sum over k of: alpha_k x A^k )

A^k = feature map of the k-th channel in the target layer

Y^c = score for class c (before softmax)

dY^c / dA^k = gradient of class score with respect to feature map
```

What Good and Bad Grad-CAM Looks Like

Good Grad-CAM (trustworthy model)	Heat concentrated on the actual lesion — the border, colour variation, and irregular texture. The model is looking at the right features.
Bad Grad-CAM (suspicious model)	Heat on hair, ruler marks, dark corners, or the healthy surrounding skin. The model learned spurious correlations, not real medical features.
Metadata Fusion improvement	The metadata branch provides clinical context, which helps the image branch focus on lesion-specific features rather than background artefacts.

Grad-CAM in This Project

This project generates Grad-CAM heatmaps for all 8 classes, with special focus on minority classes (AK, DF, SCC, VASC). It also compares SA-Net (baseline) vs Metadata Fusion model attention to show that the new model focuses better on lesion regions.

CHAPTER 19

Experimental Design

Why We Run Progressive Experiments

In research, you cannot just build the most complex model and call it done. You need to prove that each new piece actually helps. We do this with controlled experiments — adding one change at a time and measuring the effect.

The 6 Experiments

Exp 1	SA-Net + CrossEntropy	Thesis baseline. Custom CNN from scratch. No pretrained weights. No imbalance handling.	Lower bound. Shows where we started.
Exp 2	EfficientNetV2-S + CrossEntropy	Modern pretrained backbone. No other changes.	Quantifies: how much does a better backbone help?
Exp 3	EfficientNetV2-S + Focal Loss + WeightedSampler	Adds imbalance handling to the modern backbone.	Quantifies: how much does addressing imbalance help?
Exp 4	EfficientNetV2-S + Metadata Fusion + LabelSmoothing	Adds patient metadata (age, sex, site) as a second input.	Quantifies: how much does metadata improve diagnosis?
Exp 5	Final Proposed Model (complete)	All together: Fusion + Focal Loss + Sampler + MixUp + LabelSmoothing + Albumentations.	The best model. Shows peak performance.
Exp 6	ConvNeXt-Tiny + Metadata Fusion	Alternative backbone in the fusion framework.	Backbone comparison within the multimodal setting.

The Ablation Study

An ablation study is when you remove one component at a time to see how much it contributes. Like removing ingredients from a recipe to see which ones really matter. In this project, the ablation shows the Macro F1 improvement at each step:

Ablation Interpretation Guide

SA-Net Baseline → EfficientNetV2-S: measures backbone improvement

EfficientNetV2-S → + Focal Loss + Sampler: measures imbalance handling improvement

+ Focal Loss → + Metadata Fusion: measures multimodal learning improvement (the key new contribution)

+ Metadata Fusion → Full Pipeline: measures MixUp and augmentation improvement

Each delta (change) in Macro F1 is reported in the comparison table.

CHAPTER 20

Statistical Analysis

Mean and Variance in Deep Learning

Mean and variance tell you about the centre and spread of a distribution. In training, we look at the mean loss and variance of loss across batches to understand if training is stable.

```
Mean: x_bar = (1/n) x sum(x_i) — average value

Variance: sigma^2 = (1/n) x sum( (x_i - x_bar)^2 ) — spread

Standard Deviation: sigma = sqrt(sigma^2) — average distance from mean
```

Bias-Variance Trade-off

Every model has two types of error. Bias is error from wrong assumptions — the model is too simple to capture the true pattern. Variance is error from sensitivity to noise in training data — the model is too complex and memorises noise.

High Bias (Underfitting)	Model is too simple. Training accuracy is low. Validation accuracy is also low. Curves don't converge well.
High Variance (Overfitting)	Model is too complex. Training accuracy is very high. Validation accuracy is much lower. Large gap between curves.
Good Balance	Training and validation accuracy are both high and close together. Loss curves converge and stabilise. Small gap.

Why We Use Macro F1 as the Primary Metric

Macro F1 treats each class equally regardless of size. If the model gets 99% on NV but 10% on VASC, the Macro F1 is low. This forces us to build a model that is good at ALL classes, not just the easy majority ones. This is the scientifically correct primary metric for imbalanced medical classification.

CHAPTER 21

Results and Interpretation

How to Read Training Curves

Loss curve drops, then flattens	Normal healthy training. Model learned quickly then stabilised.
Training loss drops but validation loss rises	Overfitting. Model memorising training data. Try more regularisation or augmentation.
Both losses stay high	Underfitting. Model too simple or learning rate too small.
Spiky validation loss	High variance in validation batches. Normal with small validation sets.
Val F1 improves steadily then plateaus	Good training. Early stopping fires near plateau.

How to Read the Confusion Matrix

In a good confusion matrix, all the numbers should be on the diagonal (top-left to bottom-right). High numbers on the diagonal = correct predictions. High numbers off-diagonal = the model is confusing those two classes.

Common Confusions in Skin Cancer AI

MEL vs NV: Melanoma and moles look very similar. High confusion here is expected and dangerous.

AK vs BCC: Both are caused by sun damage and look similar dermoscopically.

SCC vs BCC: Both are keratinocyte cancers with similar appearances.

DF vs NV: Both are small round lesions. Common confusion in minority class models.

Interpreting ROC Curves

The ROC (Receiver Operating Characteristic) curve shows how well the model separates one class from all others at different decision thresholds. AUC (Area Under Curve) = 1.0 means perfect. AUC = 0.5 means random.

AUC > 0.95: Excellent class separation

AUC 0.85-0.95: Good performance

AUC 0.70-0.85: Fair performance — consider reviewing this class

AUC < 0.70: Poor — model struggling with this class

CHAPTER 22

Research Methodology

What Makes This a Research Project?

Not every deep learning project is research. Research means making a contribution that did not exist before, proving it works, and documenting it so others can build on it.

Literature Review	Study existing work. Understand what has been tried. Identify gaps. This project identified metadata integration as a gap in the thesis.
Problem Formulation	Define exactly what you are solving. This project: improve multi-class skin cancer classification with metadata and modern architectures.
Dataset Selection	Choose a standard benchmark (ISIC 2019) so results can be compared to other papers.
Controlled Experiments	Change one thing at a time. Report what each change does. This is the ablation study.
Rigorous Evaluation	Use multiple metrics. Test on held-out data not seen during training. Report per-class metrics.
Novel Contribution	The metadata fusion model combining EfficientNetV2-S with patient clinical data is the primary new contribution beyond the thesis.

The Research Contribution Statement

Proposed contribution: 'A metadata-aware, imbalance-aware deep learning framework for multi-class skin cancer diagnosis that fuses dermoscopic image features with patient clinical metadata, using modern CNN backbones and Focal Loss, improving minority class recall and providing clinically interpretable predictions through Grad-CAM.'

CHAPTER 23

Programming and Software Concepts

PyTorch — Why It Was Chosen

PyTorch is the most popular deep learning library in research. It uses dynamic computation graphs — you write normal Python code and PyTorch builds the graph as it runs. This makes debugging easy and experimentation fast.

Key PyTorch Concepts

Tensor	A multi-dimensional array (like a matrix but for GPU). An image is a tensor of shape [3, 224, 224].
autograd	Automatic differentiation. PyTorch tracks all operations and automatically computes gradients.
nn.Module	Base class for all neural network layers. Every model you build inherits from this.
DataLoader	Handles batching, shuffling, and parallel data loading. Essential for training efficiency.
torch.compile	PyTorch 2.0 feature. Analyses your model and creates an optimised GPU computation graph.
state_dict	A dictionary of all model weights. Used to save and load models.
Checkpoint	Saved model weights at the best validation step. Allows resuming training or deployment.

Why Modular Code Matters

In this project, each component is a separate module: preprocessing, dataset, model, training loop, evaluation. This means you can change the backbone (try ConvNeXt instead of EfficientNetV2) without touching the training code. You can add a new loss function without changing the data pipeline.

Reproducibility is enforced by setting all random seeds (Python random, NumPy, PyTorch) to the same value (42) at the start of every experiment.

CHAPTER 24

Challenges and Limitations

Class Imbalance	50:1 ratio between majority and minority classes. Addressed with Focal Loss, WeightedSampler, and LabelSmoothing but not fully solved.
Visual Similarity	MEL, NV, and BKL look almost identical to the eye and to early CNN layers. High confusion between these classes is inherent.
Dataset Bias	ISIC 2019 is mostly Fitzpatrick skin types I-III (lighter skin tones). Performance may be worse on darker skin tones.
Noisy Labels	Some images may have been labelled incorrectly by doctors who disagreed. No way to fix this without re-labelling.
Domain Shift	Model trained on dermoscope images. May not work well on smartphone photos or images from different hospitals.
Missing Metadata	2,631 images have no anatomical site. 437 have no age. Median imputation handles this but is not perfect.
Compute Cost	Training 6 experiments with 25 epochs each requires significant GPU time. Limited experimentation budget.
Generalisability	Model validated on ISIC 2019 test split only. External validation on new hospitals not performed.

CHAPTER 25

Future Improvements

Larger Datasets	Add HAM10000, ISIC 2020, and Fitzpatrick17k for more diversity and better fairness across skin tones.
Segmentation Pre-processing	Use SAM (Segment Anything Model) to isolate the lesion before classification. Removes background noise completely.
Multimodal Transformers	Replace the CNN backbone with a vision transformer that can directly incorporate metadata tokens alongside image patches.
Longitudinal Analysis	Track the same lesion over time (multiple visits). Temporal change is a powerful diagnostic signal.
Federated Learning	Train on data from multiple hospitals without sharing patient data. Better privacy and data diversity.
Diffusion Models for Augmentation	Use generative AI to create realistic synthetic minority-class images. Better than MixUp for extreme imbalance.
Clinical Deployment	Package as a mobile app or web service for dermatologists to use in clinics. Add confidence intervals.
Explainability Beyond Grad-CAM	Add SHAP values for metadata features — show which metadata feature (age? site?) contributed most to each prediction.

CHAPTER 26

Final Conclusion

What Was Built

This project built a modern, multimodal deep learning system for classifying 8 types of skin lesions from the ISIC 2019 dataset. It moved significantly beyond the original SA-Net thesis baseline by introducing metadata fusion, modern pretrained backbones, advanced augmentation, and a structured ablation study.

Summary of New Techniques

Metadata Fusion	**NEW**	Combines patient age, sex, and anatomical site with image features for better diagnosis.
EfficientNetV2-S	**NEW**	Modern NAS-designed backbone pretrained on ImageNet. Better accuracy than SA-Net.
ConvNeXt-Tiny	**NEW**	Transformer-inspired CNN backbone for backbone comparison.
Albumentations Augmentation	**NEW**	GridDistortion, ElasticTransform, CoarseDropout — richer than basic torchvision.
MixUp	**NEW**	Creates virtual training examples by blending images. Better generalisation.
Label Smoothing	**NEW**	Prevents overconfidence on majority classes.
CosineAnnealingWarmRestarts	**NEW**	Better learning rate schedule for convergence.
torch.compile + AMP	**NEW**	Maximises GPU utilisation for fast training.
Focal Loss	**KEPT**	Retained from baseline — focuses training on hard minority-class examples.
WeightedRandomSampler	**KEPT**	Retained from baseline — ensures minority classes appear in every batch.
Grad-CAM	**IMPROVED**	Now compares SA-Net vs Fusion model attention. Minority-class focus added.

SA-Net + Skin Attention Blocks	**BASELINE**	Original thesis model. Kept as Experiment 1 for comparison only.

The Core Research Contribution

Final Research Statement

'We propose a metadata-aware, imbalance-aware multimodal deep learning framework for multi-class skin cancer diagnosis. By combining dermoscopic image features from a pretrained EfficientNetV2-S encoder with patient clinical metadata (age, sex, anatomical site) in a dual-branch architecture, and training with Focal Loss and WeightedRandomSampler to address class imbalance, our model improves minority class detection performance while providing clinically interpretable predictions through Grad-CAM visualisations — directly addressing the limitations identified in the original thesis baseline.'

Key Takeaways for Your Thesis Defense

- You can explain WHY each technique was added — not just that it was added.
- You can compare your model to the baseline with clear numbers from 6 experiments.
- You can explain the clinical relevance — why MEL and SCC recall matters more than overall accuracy.
- You can explain the math behind Focal Loss, Grad-CAM, and metadata encoding if asked.
- You can discuss the limitations honestly — which is a sign of good research, not weakness.
- The ablation study shows you understand what each component contributes independently.

This project demonstrates that combining modern deep learning architectures with clinical metadata produces a more accurate and clinically trustworthy skin cancer diagnosis system — and that rigorous experimental design can clearly prove each contribution.